

TESTING, CI/CD & ENTERPRISE

Quality Pipelines, Custom Extensions,
and Centralized Automation

Professional Reference Documentation

Version 1.0

Table of Contents

PART I — TESTING & CI/CD PIPELINES

1. Why Testing Matters
2. Linting with ansible-lint
3. Testing Roles with Molecule
4. CI/CD Pipeline Integration
- 4.1 GitHub Actions
- 4.2 GitLab CI
- 4.3 Jenkins

PART II — CUSTOM MODULES & FILTER PLUGINS

5. When Built-in Tools Are Not Enough
6. Writing a Custom Module
7. Writing a Custom Filter Plugin
8. Distributing Custom Extensions

PART III — ENTERPRISE ORCHESTRATION

9. AWX and Automation Controller
10. RBAC: Role-Based Access Control
11. Credential Management
12. Scheduled Jobs and Surveys
13. Workflow Templates
14. Best Practices
15. Quick Reference

PART I — TESTING & CI/CD PIPELINES

1. Why Testing Matters

Ansible playbooks that run in production are code. Like any other code, they benefit from automated checking, isolated testing, and a continuous integration pipeline that catches regressions before they reach a managed host. A single typo in a playbook can take down a fleet of servers; a single unintended change in a role can silently break every host that consumes it.

Three layers of verification form a mature Ansible testing strategy:

- **Linting** — static analysis that catches syntax errors, style violations, and known anti-patterns without executing anything.
- **Unit / role testing** — running a role inside a disposable container to verify its behaviour in isolation.
- **Pipeline integration** — running the above automatically on every commit and gating merges on the result.

This part covers all three layers, with concrete tooling and configuration for the three most common CI platforms: GitHub Actions, GitLab CI, and Jenkins.

2. Linting with ansible-lint

ansible-lint is the de facto static analyser for Ansible content. It parses playbooks, roles, and collections, then checks them against a large set of rules covering YAML syntax, task style, deprecated keywords, and unsafe patterns.

2.1 Installation

```
# pip (recommended -- works on any platform)
python3 -m pip install --user ansible-lint

# Homebrew
brew install ansible-lint

# Verify
ansible-lint --version
```

2.2 Running the Linter

```
# Lint every playbook and role in the current directory
ansible-lint

# Lint a specific playbook
ansible-lint site.yml

# Lint a role
ansible-lint roles/nginx

# Only fail on errors (ignore warnings)
ansible-lint --strict

# Show the rules that are active
ansible-lint -L

# Output in a machine-readable format for CI
ansible-lint -f pep8
```

2.3 Common Rules

Rule	What it catches
name[play] / name[task]	Plays and tasks without a descriptive name.
yaml[truthy]	Using yes/no instead of true/false.
yaml[indentation]	Wrong indentation or tabs instead of spaces.
no-changed-when	command / shell tasks without changed_when.
command-instead-of-module	Using shell to do what a dedicated module already does.
risky-file-permissions	Files written without an explicit mode.
fqcn[action-core]	Short module names instead of fully qualified names.
jinja[spacing]	Jinja2 expressions with inconsistent spacing.
deprecated-module	Modules that have been replaced by newer versions.
literal-compare	Comparing a boolean to true or false explicitly.

Table 1 — Representative ansible-lint rules.

2.4 Configuration

Project-specific rules and exclusions are declared in a `.ansible-lint` file at the root of the repository. This lets a team suppress rules that do not fit its style while keeping the rest active.

```
# .ansible-lint
---
profile: production
exclude_paths:
  - .cache/
  - molecule/
  - tests/
warn_list:
  - experimental
  - jinja[spacing]
skip_list:
  - name[casing]      # allow Title Case in this project
  - yaml[line-length] # long lines are fine here
```

2.5 Pre-commit Hook

Running ansible-lint as a pre-commit hook catches problems before the code ever reaches CI. The hook runs on staged files only, so it is fast.

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/ansible/ansible-lint
    rev: v24.2.0
    hooks:
      - id: ansible-lint
        files: \.(yaml|yml)$

# Install the hook
pre-commit install

# Run manually against all files
pre-commit run --all-files
```

3. Testing Roles with Molecule

Molecule is the standard framework for testing Ansible roles. It provisions a disposable environment — typically a container or a virtual machine — runs the role inside it, executes assertions, and tears the environment down. Because each test starts from a clean state, results are reproducible.

3.1 Installation

```
# Install Molecule with the Docker driver (most common)
python3 -m pip install --user "molecule[docker]"

# Additional drivers for other backends
python3 -m pip install --user "molecule[podman]"
python3 -m pip install --user "molecule[vagrant]"

# Verify
molecule --version
```

3.2 Creating a Role with Molecule

```
# Create a new role with a Molecule scaffold
ansible-galaxy role init nginx --force
cd nginx
molecule init scenario --driver-name docker

# Directory layout after init:
# roles/nginx/
# |-- tasks/main.yml
# |-- handlers/main.yml
# |-- templates/
# |-- defaults/main.yml
# `-- molecule/
#     |-- default/
#         |-- molecule.yml      # scenario configuration
#         |-- converge.yml     # playbook that applies the role
#         `-- verify.yml       # assertions that validate the result
```

3.3 Scenario Configuration

```
# molecule/default/molecule.yml
---
dependency:
  name: galaxy

driver:
  name: docker

platforms:
  - name: ubuntu-22
    image: geerlingguy/docker-ubuntu2204-ansible:latest
    pre_build_image: true
    command: /lib/systemd/systemd
    privileged: true
    volumes:
      - /sys/fs/cgroup:/sys/fs/cgroup:rw
    cgroupns_mode: host

  - name: rockylinux-9
    image: geerlingguy/docker-rockylinux9-ansible:latest
    pre_build_image: true
    command: /usr/sbin/init
    privileged: true
    volumes:
      - /sys/fs/cgroup:/sys/fs/cgroup:rw
    cgroupns_mode: host

provisioner:
  name: ansible
  log: true
  options:
    v: true
  inventory:
    group_vars:
      all:
```

```
    nginx_port: 8080
```

```
verifier:
  name: ansible

scenario:
  test_sequence:
    - destroy
    - create
    - prepare
    - converge
    - idempotence
    - verify
    - destroy
```

3.4 The Converge and Verify Playbooks

```
# molecule/default/converge.yml -- applies the role
---
- name: Converge
  hosts: all
  become: true
  tasks:
    - name: Include the role under test
      ansible.builtin.include_role:
        name: nginx

# molecule/default/verify.yml -- asserts the result
---
- name: Verify
  hosts: all
  become: true
  tasks:
    - name: Check that nginx is installed
      ansible.builtin.package:
        name: nginx
        state: present
      check_mode: true
      register: pkg
      failed_when: pkg.changed

    - name: Check that nginx is running
      ansible.builtin.service:
        name: nginx
        state: started
      check_mode: true
      register: svc
      failed_when: svc.changed

    - name: Check that the configuration file exists
      ansible.builtin.stat:
        path: /etc/nginx/nginx.conf
      register: conf
      failed_when: not conf.stat.exists
```

3.5 Running the Tests

```
# Full test sequence on all platforms
molecule test

# Individual steps
molecule create      # spin up the containers
molecule converge    # run the role
molecule verify      # run assertions
molecule destroy     # tear down
molecule login       # open a shell inside the running instance

# Test a specific platform
molecule test --platform-name ubuntu-22

# Test a specific scenario
molecule test --scenario-name alternative
```

The idempotence step is critical. Molecule runs the role twice and verifies that the second run reports no changes. A role that fails idempotence is a role that will misbehave in production.

4. CI/CD Pipeline Integration

The final layer is automation: running linting and Molecule on every commit and gating merges on the result. The three most common platforms have different configuration formats, but the underlying workflow is identical.

Pipeline stages:

- **Lint** — run `ansible-lint` against every playbook and role.
- **Syntax check** — run `ansible-playbook --syntax-check`.
- **Test** — run Molecule against each role.
- **Deploy** — apply the playbook to a staging environment, then to production with a manual approval gate.

4.1 GitHub Actions

```
# .github/workflows/ansible-ci.yml
---
name: Ansible CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  lint:
    name: Lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - name: Install ansible-lint
        run: python3 -m pip install ansible-lint
      - name: Run ansible-lint
        run: ansible-lint

  molecule:
    name: Molecule
    runs-on: ubuntu-latest
    needs: lint
    strategy:
      matrix:
        role: [nginx, postgresql, redis]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
```

```
    python-version: '3.12'
  - name: Install dependencies
    run: |
      python3 -m pip install ansible "molecule[docker]" ansible-lint
  - name: Run Molecule
    run: molecule test
    working-directory: roles/${{ matrix.role }}
    env:
      PY_COLORS: '1'
      ANSIBLE_FORCE_COLOR: '1'

deploy:
  name: Deploy to production
  runs-on: ubuntu-latest
  needs: molecule
  if: github.ref == 'refs/heads/main'
  environment:
    name: production
    url: https://dashboard.example.com
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: '3.12'
    - name: Install Ansible
      run: python3 -m pip install ansible
    - name: Configure SSH
      run: |
        mkdir -p ~/.ssh
        echo "${{ secrets.SSH_PRIVATE_KEY }}" > ~/.ssh/id_ed25519
        chmod 600 ~/.ssh/id_ed25519
        echo "${{ secrets.SSH_KNOWN_HOSTS }}" > ~/.ssh/known_hosts
    - name: Write vault password
      run: |
        echo "${{ secrets.VAULT_PASSWORD }}" > /tmp/vault_pass
```

```
    chmod 600 /tmp/vault_pass
  - name: Deploy
    run: |
      ansible-playbook -i inventory/production site.yml \
        --vault-password-file /tmp/vault_pass
```

4.2 GitLab CI

```
# .gitlab-ci.yml
---
stages:
  - lint
  - test
  - deploy

variables:
  ANSIBLE_FORCE_COLOR: "1"
  PIP_CACHE_DIR: "${CI_PROJECT_DIR}/.cache/pip"

cache:
  paths:
    - .cache/pip/

default:
  image: python:3.12-slim
  before_script:
    - python3 -m pip install --upgrade pip
    - python3 -m pip install ansible ansible-lint "molecule[docker]"

lint:
  stage: lint
  script:
    - ansible-lint
    - ansible-playbook --syntax-check -i inventory/staging site.yml

molecule:
  stage: test
  parallel:
    matrix:
      - ROLE: [nginx, postgresql, redis]
  script:
    - cd roles/$ROLE
    - molecule test
```

```
deploy-staging:
  stage: deploy
  environment:
    name: staging
  script:
    - echo "$VAULT_PASSWORD" > /tmp/vault_pass
    - chmod 600 /tmp/vault_pass
    - ansible-playbook -i inventory/staging site.yml
      --vault-password-file /tmp/vault_pass
  only:
    - develop

deploy-production:
  stage: deploy
  environment:
    name: production
  script:
    - echo "$VAULT_PASSWORD" > /tmp/vault_pass
    - chmod 600 /tmp/vault_pass
    - ansible-playbook -i inventory/production site.yml
      --vault-password-file /tmp/vault_pass
  when: manual
  only:
    - main
```

4.3 Jenkins

```
// Jenkinsfile
pipeline {
    agent any

    environment {
        ANSIBLE_FORCE_COLOR = '1'
        VAULT_PASSWORD      = credentials('ansible-vault-password')
        SSH_KEY              = credentials('deploy-ssh-key')
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Lint') {
            steps {
                sh '''
                    python3 -m pip install --user ansible-lint
                    ansible-lint
                '''
            }
        }

        stage('Syntax Check') {
            steps {
                sh 'ansible-playbook --syntax-check -i inventory/staging site.yml'
            }
        }

        stage('Molecule') {
            steps {
                sh '''
```

```

        python3 -m pip install --user "molecule[docker]"
        cd roles/nginx && molecule test
    ...
}
}

stage('Deploy to Staging') {
    steps {
        withCredentials([file(credentialsId: 'vault-password-file',
                                variable: 'VAULT_FILE')]) {
            sh '''
                ansible-playbook -i inventory/staging site.yml \
                --vault-password-file "$VAULT_FILE"
            ...
        }
    }
}

stage('Deploy to Production') {
    input {
        message 'Deploy to production?'
        ok 'Deploy'
    }
    steps {
        withCredentials([file(credentialsId: 'vault-password-file',
                                variable: 'VAULT_FILE')]) {
            sh '''
                ansible-playbook -i inventory/production site.yml \
                --vault-password-file "$VAULT_FILE"
            ...
        }
    }
}

}

post {
    always {
        cleanWs()
    }
    failure {
        slackSend channel: '#ops',
                  message: "Pipeline failed: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
    }
}
}

```

PART II — CUSTOM MODULES & FILTER PLUGINS

5. When Built-in Tools Are Not Enough

Ansible ships with hundreds of modules and thousands more are available through Galaxy collections. Even so, specific infrastructure occasionally requires behaviour that no existing module provides: an internal API without a published client, a proprietary appliance, or a domain-specific calculation that is repeated across many playbooks.

Two extension mechanisms cover almost every case:

- **Custom modules** — Python programs that perform an action and return JSON. They are the right choice when the logic must run on the managed node.
- **Custom filter plugins** — Python functions that transform data inside a Jinja2 expression. They are the right choice when the logic is a data transformation on the control node.

Both are distributed inside Ansible collections, which package modules, filters, roles, and documentation together.

6. Writing a Custom Module

A custom module is a standalone Python file that reads arguments from a JSON file, performs its work, and prints a JSON result to standard output. Ansible handles argument parsing and result formatting through the `AnsibleModule` helper class.

6.1 Minimal Module Structure

```
#!/usr/bin/python
# library/my_service_status.py

from ansible.module_utils.basic import AnsibleModule

def get_service_status(name):
    # In a real module this would query the service, an API, or a file.
    # This stub returns a fixed result for illustration.
    return {"name": name, "running": True, "uptime_seconds": 3600}

def main():
    module = AnsibleModule(
        argument_spec=dict(
            name=dict(type="str", required=True),
            timeout=dict(type="int", default=10),
        ),
        supports_check_mode=True,
    )

    name = module.params["name"]

    if module.check_mode:
        module.exit_json(changed=False, **get_service_status(name))

    try:
        status = get_service_status(name)
    except Exception as exc:
        module.fail_json(msg="Failed to query service: %s" % exc)

    module.exit_json(changed=False, **status)

if __name__ == "__main__":
```

```
    main()
```

6.2 Calling the Module from a Playbook

```
# library/ is searched automatically relative to the playbook
- name: Query the status of a custom service
  my_service_status:
    name: myapp
    timeout: 5
  register: result

- name: Show the status
  ansible.builtin.debug:
    msg: "{{ result.name }}" running={{ result.running }}
```

6.3 Key AnsibleModule Features

Feature	Purpose
argument_spec	Declare typed arguments and defaults. Required arguments are enforced automatically.

Feature	Purpose
<code>supports_check_mode</code>	Allow the module to run in <code>--check</code> mode.
<code>module.params</code>	Dictionary of validated, typed arguments.
<code>module.exit_json(...)</code>	Return a successful result as JSON.
<code>module.fail_json(msg=...)</code>	Return a failure with a message.
<code>module.run_command(...)</code>	Execute a command safely (no shell injection).
<code>module.check_mode</code>	Boolean indicating whether <code>--check</code> was used.
<code>module.warn(...)</code>	Emit a warning without failing.

Table 2 — Essential `AnsibleModule` attributes and methods.

6.4 Example: A Module That Manages an API Resource

```
#!/usr/bin/python
# library/internal_api_user.py

from ansible.module_utils.basic import AnsibleModule
from ansible.module_utils.urls import fetch_url
import json

def api_request(module, method, url, payload=None):
    headers = {
        "Content-Type": "application/json",
        "Authorization": "Bearer %s" % module.params["token"],
    }
    body = json.dumps(payload).encode("utf-8") if payload else None
    response, info = fetch_url(module, url, method=method,
                              headers=headers, data=body, timeout=30)
    if info["status"] >= 400:
        module.fail_json(msg="API error %s: %s" %
                        (info["status"], info.get("body", "")))
    return json.loads(response.read())

def main():
    module = AnsibleModule(
        argument_spec=dict(
            username=dict(type="str", required=True),
            email=dict(type="str", required=True),
            token=dict(type="str", required=True, no_log=True),
            endpoint=dict(type="str", default="https://api.internal.example.com"),
            state=dict(type="str", default="present",
                      choices=["present", "absent"]),
        ),
        supports_check_mode=True,
    )
```

```
username = module.params["username"]
endpoint = module.params["endpoint"]
state = module.params["state"]
url = "%s/users/%s" % (endpoint, username)

if state == "absent":
    if module.check_mode:
        module.exit_json(changed=True, msg="Would delete %s" % username)
    api_request(module, "DELETE", url)
    module.exit_json(changed=True, msg="Deleted %s" % username)

payload = {"username": username, "email": module.params["email"]}

try:
    existing = api_request(module, "GET", url)
except SystemExit:
    existing = None

if existing and existing.get("email") == payload["email"]:
    module.exit_json(changed=False, msg="User already correct")

if module.check_mode:
    module.exit_json(changed=True, msg="Would create or update")

api_request(module, "PUT", url, payload)
module.exit_json(changed=True, msg="User created or updated",
                  user=payload)

if __name__ == "__main__":
    main()
```

The `no_log=True` flag on the `token` argument ensures the value is masked in output and logs. Every argument that carries a secret must use this flag.

7. Writing a Custom Filter Plugin

A filter plugin is a Python function registered with Jinja2. It can be used anywhere a built-in filter can, and it operates on the control node during template rendering or playbook execution. Filters are ideal for data transformation that would otherwise require a long chain of built-in filters.

7.1 Simple Filter Example

```
# filter_plugins/network_filters.py

def cidr_to_netmask(cidr):
    """Convert a CIDR prefix length (24) to a dotted netmask (255.255.255.0)."""
    cidr = int(cidr)
    if not 0 <= cidr <= 32:
        raise ValueError("CIDR must be between 0 and 32")
    bits = (0xffffffff << (32 - cidr)) & 0xffffffff
    return ".".join(str((bits >> shift) & 0xff) for shift in (24, 16, 8, 0))

def parse_kv_string(value, sep=",", kv_sep="="):
    """Turn 'a=1,b=2' into {'a': '1', 'b': '2'}."""
    return dict(item.split(kv_sep, 1) for item in value.split(sep) if item)

class FilterModule(object):
    def filters(self):
        return {
            "cidr_to_netmask": cidr_to_netmask,
            "parse_kv_string": parse_kv_string,
        }
```

7.2 Using the Filter

```
- name: Show the netmask for each subnet
  ansible.builtin.debug:
    msg: "10.0.0.0/{{ item }} -> {{ item | cidr_to_netmask }}"
  loop: [24, 25, 26]

- name: Parse an options string
  ansible.builtin.debug:
    msg: "{{ 'timeout=30,retries=3' | parse_kv_string }}"
  # prints: {'timeout': '30', 'retries': '3'}
```

7.3 Where Filter Plugins Live

- `filter_plugins/` next to the playbook — searched automatically.
- `roles/<role>/filter_plugins/` — available to that role only.
- `collections/<ns>/<name>/plugins/filter/` — packaged inside a collection.

Filter plugins are also used by Ansible internally. When you write your own, use a distinctive prefix on the function name to avoid collisions with built-in filters or with filters from other collections.

8. Distributing Custom Extensions

Custom modules and filters should be packaged inside an Ansible collection. A collection is a directory tree with a well-defined layout that Ansible can install, version, and share through Galaxy or a private repository.

```
# Create a new collection skeleton
ansible-galaxy collection init acme.infrastructure
cd acme/infrastructure

# Resulting layout:
# acme/infrastructure/
# |-- galaxy.yml           # collection metadata
# |-- README.md
# |-- plugins/
# |   |-- modules/
# |   |   |-- my_service_status.py
# |   |-- filter/
# |       |-- network_filters.py
# |-- roles/
# |-- docs/

# Build and publish
ansible-galaxy collection build
ansible-galaxy collection publish acme-infrastructure-1.0.0.tar.gz

# Install from Galaxy
ansible-galaxy collection install acme.infrastructure

# Install from a private Git repository
ansible-galaxy collection install \
    git+https://git.example.com/ansible/acme-infrastructure.git
```

Referencing collection content in a playbook:

```
# Fully qualified names for modules and filters
- name: Use a custom module from the acme.infrastructure collection
  acme.infrastructure.my_service_status:
    name: myapp

- name: Use a custom filter
  ansible.builtin.debug:
    msg: "{{ 24 | acme.infrastructure.cidr_to_netmask }}"
```

PART III — ENTERPRISE ORCHESTRATION

9. AWX and Automation Controller

AWX is the open-source upstream project for Red Hat Ansible Automation Controller. Both provide a web UI, REST API, and job scheduler on top of Ansible. The CLI remains available underneath, but the platform adds centralized control, audit trails, and role-based access.

9.1 Why Move Beyond the CLI

- **Centralised credentials.** SSH keys, vault passwords, and API tokens are stored encrypted in the platform rather than on individual engineer workstations.
- **Audit trail.** Every job run is logged with the user, timestamp, playbook revision, and full output.
- **Access control.** Users and teams are granted fine-grained permissions on specific inventories, projects, and templates.
- **Scheduling.** Jobs run on a cron-like schedule without needing an external orchestrator.
- **Surveys.** Non-technical users launch jobs by filling in a form, without ever touching YAML.
- **Workflows.** Multiple job templates chain together, with success and failure branches, into complex automations.
- **Notifications.** Built-in integration with Slack, email, PagerDuty, and webhooks.

9.2 Core Objects

Object	Purpose
Organization	Top-level container for users, teams, and resources.
User	Individual account with credentials and permissions.
Team	A group of users that receives permissions collectively.
Credential	Encrypted secret (SSH key, vault password, cloud token).
Project	A Git repository containing playbooks and roles.
Inventory	A static file, dynamic source, or smart inventory.
Job Template	Playbook + inventory + credential + extra vars. The unit of execution.
Workflow Template	A graph of job templates with success / failure branches.
Schedule	Cron-like rule that launches a job template.
Survey	Form-based prompt for extra variables at launch time.
Notification	Webhook, Slack, or email target triggered by job events.

Table 3 — Core objects in Automation Controller.

10. RBAC: Role-Based Access Control

Automation Controller implements a role-based permission model. Every object (project, inventory, credential, job template) has a set of roles, and each role grants a set of permissions. Users and teams are assigned roles on objects.

10.1 Standard Roles

- **Admin** — full control over the object, including deletion and permission changes.
- **Execute** — may run jobs that use the object but may not modify it.
- **Use** — may reference the object in job templates.
- **Read** — may view the object but not act on it.
- **Update** — may modify the object.
- **Approver** — may approve or deny workflow steps that require approval.

10.2 Example Permission Model

A typical enterprise deployment uses organizations to separate business units, teams to group engineers, and role assignments to grant the minimum necessary access.

```
Organization: Platform Engineering
■■■ Team: Network Engineers
■ ■■■ Execute on Inventory: datacenter-network
■ ■■■ Execute on Project: network-automation
■ ■■■ Use on Credential: network-device-ssh
■■■ Team: Linux Administrators
■ ■■■ Execute on Inventory: linux-fleet
■ ■■■ Execute on Project: linux-baseline
■ ■■■ Approver on Workflow: patch-management
■■■ Team: Automation Developers
■■■ Admin on Project: shared-playbooks
■■■ Read on all Inventories
```

10.3 Creating RBAC Objects via the API

```
# Create a team
curl -X POST https://controller.example.com/api/v2/teams/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name": "Linux Administrators", "organization": 1}'

# Grant Execute on an inventory to that team
curl -X POST https://controller.example.com/api/v2/teams/5/roles/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "id": 3,
    "role_field": "execute_role",
    "content_type": "inventory",
    "object_id": 7
  }'
```

11. Credential Management

Credentials in Automation Controller are encrypted at rest and decrypted only in memory during job execution. They are never exposed in job output or in the API. This is the primary advantage over storing SSH keys and vault passwords on individual workstations.

11.1 Credential Types

Type	Stores
Machine	SSH username, private key, passphrase, or password. Also sudo password.
Source Control	Git username, password, or SSH key for private repositories.
Vault	The vault password for decrypting ansible-vault files.
Amazon Web Services	Access key ID and secret, or an assumed role.
Microsoft Azure Resource Manager	Subscription ID, client ID, client secret, tenant ID.
Google Compute Engine	Service account JSON or OAuth token.
Network	Credentials for network devices (SSH, SNMP, API).
Custom	User-defined fields for internal systems.

Table 4 — Common credential types.

11.2 Injecting Credentials into Jobs

When a job template runs, the assigned credentials are decrypted and made available to Ansible as extra variables. A Machine credential supplies SSH access; a Vault credential supplies the vault password; a cloud credential populates environment variables or extra vars expected by the cloud modules.

```
# The platform injects vault credentials as if you had run:
ansible-playbook site.yml --vault-password-file /run/secrets/vault_pass

# And machine credentials as if you had run:
ansible-playbook site.yml \
  -u deploy \
  --private-key /run/secrets/ssh_key
```

12. Scheduled Jobs and Surveys

12.1 Schedules

A schedule attaches a cron-like rule to a job template. Once per hour, once per night, or on the first Monday of every month — the platform launches the job automatically and records the result.

```
# Create a schedule that runs nightly at 02:00 UTC
curl -X POST https://controller.example.com/api/v2/job_templates/12/schedules/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Nightly patch cycle",
    "rrule": "DTSTART:20260101T020000Z RRULE:FREQ=DAILY;INTERVAL=1"
  }'
```

Schedules accept the iCalendar RRULE format, which supports complex patterns such as "every second Tuesday" or "the last business day of the month".

12.2 Surveys

A survey turns a job template into a form. The playbook author defines a set of questions — text, multiple choice, or numeric — and users answer them at launch time. Answers are passed to the playbook as extra variables. This lets non-technical staff run carefully scoped automation without editing YAML.

```
# Survey specification (JSON)
{
  "name": "Release deployment",
  "description": "Deploy a release to a chosen environment",
  "spec": [
    {
      "question_name": "Version",
      "variable_name": "app_version",
      "type": "text",
      "required": true,
      "default": "v1.0.0"
    },
    {
      "question_name": "Environment",
      "variable_name": "environment",
      "type": "multiplechoice",
      "choices": ["staging", "production"],
      "required": true,
      "default": "staging"
    },
    {
      "question_name": "Restart services",
      "variable_name": "restart_services",
      "type": "multiplechoice",
      "choices": ["yes", "no"],
      "default": "yes"
    }
  ]
}
```

Surveys pair well with RBAC: grant a team Execute on a narrowly scoped job template, and members of that team can run releases without ever touching the underlying playbook.

13. Workflow Templates

A workflow template is a directed graph of job templates. Each node is a job template; edges represent the order of execution and the conditions under which the next node runs. A workflow can branch on success,

branch on failure, require manual approval, or run nodes in parallel.

13.1 A Typical Deployment Workflow



13.2 Node Types

- **Job** — runs a job template.
- **Approval** — pauses the workflow until an authorised user approves or denies it.
- **Project Sync** — refreshes a project from Git.
- **Inventory Source Sync** — refreshes a dynamic inventory.
- **Workflow Job Template** — launches another workflow as a subgraph.

13.3 Creating a Workflow via the API

```
# Create the workflow template
curl -X POST https://controller.example.com/api/v2/workflow_job_templates/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Release deployment",
    "organization": 1,
    "survey_enabled": true,
    "allow_simultaneous": false
  }'

# Add the first node (provisioning job)
curl -X POST \
  https://controller.example.com/api/v2/workflow_job_templates/8/nodes/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "unified_job_template": 12,
    "identifier": "provision",
    "all_parents_must_converge": false
  }'

# Add a child node that runs only on success
curl -X POST \
  https://controller.example.com/api/v2/workflow_job_templates/8/nodes/provision/success_nodes/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "unified_job_template": 13,
    "identifier": "configure",
    "all_parents_must_converge": false
  }'
```

14. Best Practices

Testing & CI/CD

- **Run ansible-lint on every commit.** Wire it into pre-commit and into every CI pipeline.
- **Test every role with Molecule.** Include the idempotence step; a role that fails it is not production-ready.
- **Test against multiple platforms.** Use the Molecule platforms list to cover every OS in your fleet.
- **Gate merges on green pipelines.** A playbook that fails lint or Molecule should never reach main.
- **Require manual approval for production.** Use environment protection rules or a Jenkins input step.
- **Store vault passwords in the platform secret store.** Never in the repository or in CI environment variables that outlive the build.

Custom Modules & Filters

- **Reach for a custom module only after exhausting existing collections.** Community content is well-tested and maintained.
- **Use `no_log=True` on every sensitive argument.** This masks the value in output and logs.
- **Respect check mode.** A module with `supports_check_mode=True` must not modify the system when it is enabled.
- **Return meaningful JSON.** Include enough data for a user to understand what happened, not just a boolean.
- **Prefix custom filter names.** Names like `acme_parse_config` avoid collisions with future built-in filters.
- **Package everything as a collection.** Version it, document it, and publish it so it can be reused across teams.

Enterprise Orchestration

- **Model organisations around business units.** Do not put every user in a single global organisation.
- **Assign the minimum necessary role.** Prefer Execute over Admin, and prefer team-based over user-based permissions.
- **Use credential types, not plaintext extra vars.** If a secret is needed, create a credential for it.
- **Use surveys for routine operations.** They let non-technical users run scoped automation safely.
- **Prefer workflow templates over mega-playbooks.** Small, reusable job templates chained together are easier to maintain and audit.
- **Add approval nodes before production changes.** The workflow graph is the right place for human-in-the-loop gates.
- **Configure notifications.** Alert on failure so problems are caught quickly.
- **Export configuration as code.** Use the `awxkit` or `ansible.controller` collection to keep objects in Git.

15. Quick Reference

Testing & CI/CD

Task	Command / File
Install the linter	<code>pip install ansible-lint</code>
Lint the repository	<code>ansible-lint</code>
Lint with strict mode	<code>ansible-lint --strict</code>
Project configuration	<code>.ansible-lint</code>
Install Molecule	<code>pip install "molecule[docker]"</code>
Scaffold a Molecule scenario	<code>molecule init scenario --driver-name docker</code>
Run the full test sequence	<code>molecule test</code>
Run a single step	<code>molecule converge / verify / destroy</code>
Open a shell in a running instance	<code>molecule login</code>
GitHub Actions workflow	<code>.github/workflows/ansible-ci.yml</code>
GitLab CI pipeline	<code>.gitlab-ci.yml</code>
Jenkins pipeline	<code>Jenkinsfile</code>

Table 5 — Testing and CI/CD quick reference.

Custom Extensions

Task	Location / Command
Custom module (project)	<code>library/my_module.py</code>
Custom filter (project)	<code>filter_plugins/my_filters.py</code>
Custom module (collection)	<code>plugins/modules/my_module.py</code>
Custom filter (collection)	<code>plugins/filter/my_filters.py</code>
Create a collection	<code>ansible-galaxy collection init ns.name</code>
Build a collection	<code>ansible-galaxy collection build</code>
Install from Galaxy	<code>ansible-galaxy collection install ns.name</code>
Install from Git	<code>ansible-galaxy collection install git+https://...</code>
Mask a sensitive argument	<code>token=dict(no_log=True)</code>
Support check mode	<code>supports_check_mode=True</code>

Table 6 — Custom extensions quick reference.

Enterprise Orchestration

Task	Where
Create an organisation	UI: Organizations → Add / API: POST <code>/api/v2/organizations/</code>
Create a team	UI: Teams / API: POST <code>/api/v2/teams/</code>
Grant a role to a team	UI: Team → Roles / API: POST <code>/api/v2/teams/N/roles/</code>

Task	Where
Create a credential	UI: Credentials / API: POST /api/v2/credentials/
Create a project	UI: Projects / API: POST /api/v2/projects/
Create a job template	UI: Templates / API: POST /api/v2/job_templates/
Attach a schedule	UI: Template → Schedules / API: .../schedules/
Define a survey	UI: Template → Survey / API: .../survey_spec/
Build a workflow	UI: Templates → Workflow / API: POST /api/v2/workflow_job_templates/
Export configuration as code	awxkit or ansible.controller collection
Launch a job via API	POST /api/v2/job_templates/N/launch/
View job output	GET /api/v2/jobs/N/stdout/?format=txt

Table 7 — Enterprise orchestration quick reference.

This document describes Ansible Core 2.14 and later, ansible-lint 24.x, Molecule 24.x, AWX, and Red Hat Ansible Automation Controller. Tool versions and API endpoints evolve; always consult the official documentation for the version in use.